Bazar.com - Design Document

Distributed and Operating Systems, Fall 2026 | Dana Samaro |https://github.com/danasamaro/bazar-store **1.**

Overview

Bazar.com is a tiny online book store (four books) built as three independent Flask microservices. Each runs in its own Docker container, managed by Docker Compose, and they communicate only through HTTP REST calls with JSON bodies.

```
Client --> Frontend :5000 --> Catalog :5001 (search, info)
  \--> Order :5002 --> Catalog :5001 (info, update)
```

Service	Port	Responsibility	Storage
Frontend	5000	Single entry point; forwards requests, holds no data	none
Catalog	5001	Books (id, title, topic, quantity, price); query by topic/item, update	SQLite
Order	5002	Purchases: verifies stock, decrements it, logs the order	CSV file

2. How it works

REST interface. Frontend: `GET /search/<topic>`, `GET /info/<id>`, `POST|PUT /purchase/<id>`. Catalog: the same search and info, plus `PUT /update/<id>` with a JSON body such as {"price": 55} and/or {"quantity_delta": 10}. Order: `POST|PUT /purchase/<id>`. All replies are JSON. Status codes: 200 success, 404 unknown item, 409 out of stock, 503 a downstream service is unreachable.

search / info. The frontend logs the request and forwards it to the catalog. search returns `[{"id", "title"}]` for a (case-insensitive) topic; info returns `{"title", "quantity", "price"}`.

purchase. The frontend forwards to the order server, which (1) queries the catalog with GET /info and returns 404 if the book does not exist; (2) returns 409 and logs purchase FAILED if quantity is 0; (3) calls PUT /update with quantity_delta = -1; (4) appends timestamp, item_id, title to orders.csv; (5) prints bought book '<name>' and replies success.

Concurrency. Flask runs with threaded=True, so every request gets its own thread. In the catalog each request opens its own SQLite connection and the stock change is one atomic statement, `UPDATE ... SET quantity = quantity + ? WHERE id = ? AND quantity + ? >= 0`, so stock can never go negative. In the order server a `threading.Lock` makes check-stock, decrement and log one critical section.

Persistence and deployment. docker-compose.yml mounts ./data/catalog and ./data/order as volumes, so the database and order log survive restarts. Services find each other by name on the compose network, and all addresses and paths come from environment variables (CATALOG_URL, ORDER_URL, DB_PATH, ORDERS_FILE), so the three services can run on three different machines by changing only those values.

Logging and testing. Each service prints tagged log lines ([FRONTEND], [CATALOG], [ORDER]); the catalog prints the data returned by search and info. client_tests.py runs all operations (including 404 and out-of-stock cases) and saves the client results plus per-service logs to docs/output.txt.

3. Design decisions and tradeoffs

• **Flask** - lightweight, gives REST routing, JSON and threading with very little code (as the lab recommends). • **SQLite + CSV** - the catalog needs atomic read-modify-write updates, which SQLite provides without a database server; the order log is append-only, so a CSV file is enough. Heavy databases were avoided on purpose. • **Atomic update + lock** - the catalog protects the invariant 'stock >= 0' even if clients call update directly; the order lock keeps purchases clean. Tradeoff: the lock is held during two HTTP calls, so purchases are processed one at a

time. Acceptable for a four-book store, and simple.

• **Thin frontend** - no business logic, so tiers stay independent and features like caching can be added later. • **Environment-variable configuration** - nothing is hard-coded, enabling multi-machine deployment. Both PUT and POST are accepted for purchase because the lab leaves it open.

4. Known limitations

• The lock protects only one order-server process. With several instances, check-then-decrement could race (negative stock is still prevented by the catalog, but a buyer may get 409 'could not update stock'). • No cross-service transaction: if the order server crashes (or the CSV write fails) after the stock was decremented, stock is reduced without an order record and is not rolled back.

• depends_on controls start order, not readiness, so requests in the first seconds after start-up may get 503. • Flask development server, no authentication or HTTPS, minimal input validation (e.g. price is not range-checked), exact-match topic search only, and no retries on failed back-end calls.

5. Possible improvements

• **Production setup:** gunicorn behind a reverse proxy instead of the dev server.

• **Caching** of search/info in the frontend, invalidated on purchase or update.

• **Replication and load balancing:** several catalog/order instances behind a load balancer, with a shared database (e.g. PostgreSQL) and a database transaction or distributed lock instead of the in-process lock. • **Reliable orders:** write the order as 'pending', decrement stock, then mark 'confirmed', so crashes can be recovered; a message queue could decouple order and catalog.

• **Security and health:** authentication on /update, input validation, /health endpoints with Compose healthchecks.

6. How to run

Requirements: Docker Desktop (with Compose), Python 3 (test client only) and git.

```
git clone https://github.com/danasamaro/bazar-store.git
cd bazar-store
docker compose up --build -d # frontend :5000, catalog :5001, order :5002 python
client_tests.py # runs all requests, writes docs/output.txt curl
http://localhost:5000/search/distributed%20systems
curl http://localhost:5000/info/2
curl -X POST http://localhost:5000/purchase/2
docker compose down # stop (delete ./data to reset the data)
```

Data is stored in data/catalog/catalog.db and data/order/orders.csv. Source code is in catalog/, order/ and frontend/; the sample run is in docs/output.txt.